

Product and application architecture

A fantasy football platform combining roster tools, player research, and league data.

Platform architecture

- The main application uses Next.js 16, React 19, and TypeScript. App Router pages combine server-rendered data views with client-side interactive controls.
- Tailwind CSS 4 provides styling; Radix UI primitives and Lucide icons support the component interface.
- Supabase provides PostgreSQL storage and authentication. Server and browser clients are separated, with server auth sessions read and refreshed through cookies.
- Next.js Server Actions handle authenticated mutations, including league creation, roster edits, trade offers, account sync, and membership checkout.

Fantasy product features

- League creation and joining, roster-slot management, trade blocks, trade offers, and team review tools.
- Player rankings, profiles, weekly game logs, historical trends, head-to-head comparison, and scoring-format preferences.
- Start/sit guidance and trade valuation use visible, rule-based scoring factors such as recent production, injuries, and positional scarcity; these are not trained ML models.
- Waiver targets, injury tracking, an RSS-based news feed, fantasy video cards, a player market view, community discussions, and direct messaging.

Application design

- The UI is organized into route-level dashboard pages, shared components, typed domain models, and reusable server-side data access.
- Some player profile and weekly history data is prepared as JSON for the app; user, league, and reference records are stored in Supabase.

Data engineering and football analytics

An offline NFL data workflow turns play-level sources into profile and matchup-analysis datasets.

NFL data ingestion

- R scripts use `nflreadr` to fetch weekly player stats, play-by-play, participation, Next Gen Stats receiving metrics, and FTN charting where available.
- Each dataset is saved as a compressed, season-scoped RDS file under `data/nflverse/raw`, with row counts, fetch status, and timestamps recorded in a manifest.
- The raw research files are kept outside Git because of their size. Missing or unpublished upstream seasons are recorded and skipped rather than silently treated as complete.

Matchup split analysis

- The R analysis filters regular-season pass attempts with identified receivers, then joins participation and optional charting records by game and play IDs.
- It groups receiver outcomes by coverage family/shell, blitz bucket, offensive and defensive personnel, and detailed matchup context.
- Outputs include targets, receptions, receiving yards and touchdowns, average air yards, catch rate, and yards per target. Counts accompany rates so small samples remain visible.
- The analysis is descriptive: historical associations are not causal effects, opponent-adjusted projections, or pregame predictions.

Profiles and app integration

- A separate R exporter joins player biographies and weekly statistics, preferring stable NFL GSIS IDs and retaining several seasons of trend data.
- A Node export step packages matchup summaries into player-keyed JSON, which roster analytics can join to synced teams.
- This separates large raw-data processing and research outputs from the web request path; the research pipeline does not itself update live recommendations.

Integrations, security, and delivery

User-owned data stays behind database policies while external league and content data is imported server-side.

Backend and access control

- Supabase PostgreSQL stores accounts, leagues, memberships, roster slots, trade data, subscriptions, and imported roster references.
- Row-level security policies govern reads and writes. Authenticated users can modify only their own profile, membership, and roster records; reference datasets are read-only to app users.
- Server Actions validate submitted values (including Zod schemas where used), verify the signed-in user, and revalidate affected pages after successful changes.

External services and scheduled data

- Sleeper sync uses its public API and imports rosters read-only. ESPN private-league sync is also read-only; session cookies are encrypted with AES-256-GCM before storage and kept owner-scoped.
- Fantasy news is ingested from publisher RSS feeds on an hourly GitHub Actions schedule, with publisher attribution and article links. Video metadata is refreshed through a separate fetch workflow.
- Stripe checkout and signature-verified webhooks manage paid membership status; privileged service credentials are server-side secrets, not browser variables.

Deployment and engineering scope

- The standard development and production path uses Next.js. An alternate Vite/vinext configuration targets Cloudflare Workers and is documented as a beta path requiring deployment smoke tests.
- The project provides ESLint and production build commands. package.json does not define an automated test script.
- Data availability, third-party API changes, limited sample sizes, and upstream revisions are important operational constraints. Research data sources require attribution and license-aware use.